

universidade de aveiro



deti

departamento de eletrónica,
telecomunicações e informática

Complements of Machine Learning

2025/2026



Explainable AI

Today's Agenda

- Why Explainability Matters
- The Interpretability Landscape
- Model-Agnostic Methods (LIME, SHAP)
- GradCAM: How It Works
- Limitations & When to Trust Explanations

Learning Objectives

By the end of this session, you will:

- 1. Articulate** why model explainability matters in real-world applications
- 2. Distinguish** between model-agnostic and vision-specific XAI methods
- 3. Explain** the GradCAM algorithm and implement it conceptually
- 4. Critically evaluate** when XAI explanations are reliable vs misleading
- 5. Apply** XAI thinking to your own models from previous weeks

Pre-Class Reading

Did you read the GradCAM paper?

Key concepts to have seen:

- Gradient-weighted Class Activation Mapping
- Using gradients + feature maps for localization
- Comparison with prior visualization methods (CAM, deconvolution)

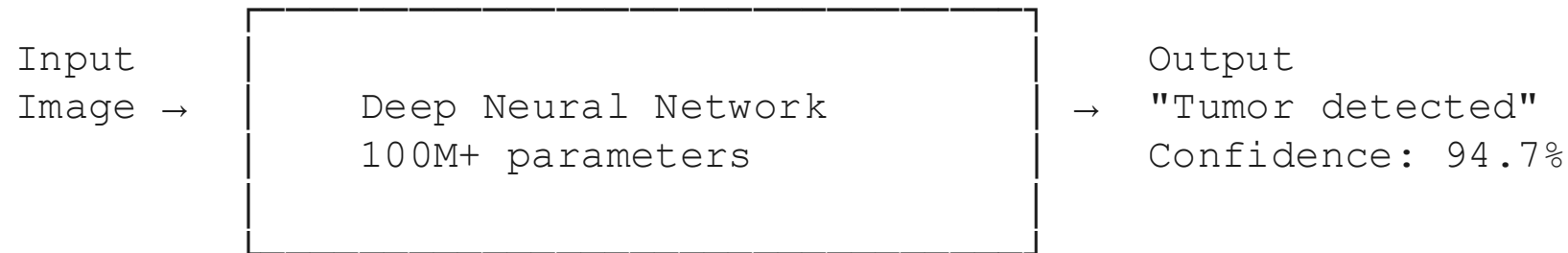
Paper: Selvaraju et al. (2017):

"Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization"

1. Why Explainability Matters

Opening the Black Box

The Black Box Problem



Doctor: "Why does the model think this is a tumor?"
Model: "..."

We built powerful models in previous classes.

But can you explain WHY they make a decision?

When Black Boxes Are Not Enough

Healthcare

Model: "Pneumonia detected — confidence 92%"

Radiologist needs to know:

- Is it looking at the lung?
- Or at the hospital label in the corner?

Finance

Model: "Loan denied"

Regulation requires:

- Explanation of factors
- EU GDPR: "right to explanation"

Autonomous Driving

Model: "No pedestrian detected" ← Wrong!

Post-incident investigation:

- What was the model focused on?
- Misled by lighting conditions?

Criminal Justice

Model: "High recidivism risk"

Ethical requirement:

- Is the model using race/ZIP proxy?
- Are explanations revealing biases?

Four Reasons to Explain Models

Reason	Description	Example
 Debugging	Find model failures before deployment	Model uses watermark, not content
 Trust	Build confidence in model predictions	Doctor verifies model looks at lesion
 Compliance	Meet regulatory requirements	GDPR "right to explanation"
 Discovery	Learn new patterns from the model	Model finds biomarker humans missed

The Clever Hans Effect

In 1900s Germany, a horse named "Clever Hans" could apparently do math.

Trainer: "What is $3 + 2$?"

Hans: *taps hoof 5 times*

Crowd: "Amazing!"

Investigation revealed: Hans was reading the TRAINER'S body language, not doing math.

He tapped until the trainer's posture relaxed.



The Clever Hans Effect

The AI version:

Model trained to classify
"wolf" vs "husky":

 on snow → Prediction: Wolf

 on grass → Prediction: Husky

What the model actually learned:

Snow background → "Wolf"

Grass background → "Husky"



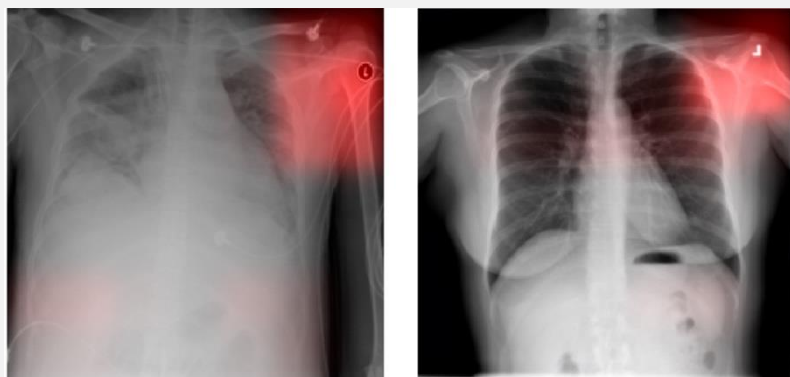
The model NEVER learned to distinguish the animals!

XAI helps us catch these "Clever Hans" moments.

Right for the Wrong Reasons – Examples

Case 1: Chest X-ray Classification

Model: 96% accuracy on test set! 🎉



<https://doi.org/10.1371/journal.pmed.1002683>

XAI revealed: Model learned to detect the HOSPITAL LABEL on the X-ray, not the actual pathology.

Different hospitals → different diseases
→ label = diagnosis proxy

Case 2: Tank Detection (Urban Legend, Real Lesson)

AI trained to detect tanks in photos.
Perfect accuracy on training set.

Reality:

"Tank" photos → all taken on cloudy days

"No tank" photos → all sunny

Model learned:

weather classifier, not tank detector.

These models would FAIL in production — XAI reveals why.

2. The Interpretability Landscape

A Map of XAI Methods

Interpretability vs Explainability

INTERPRETABILITY

(inherent property)

The model is simple enough to understand directly:

- Decision Tree
- Linear Regression
- Rule-based systems

"I can read the model"

EXPLAINABILITY

(post-hoc property)

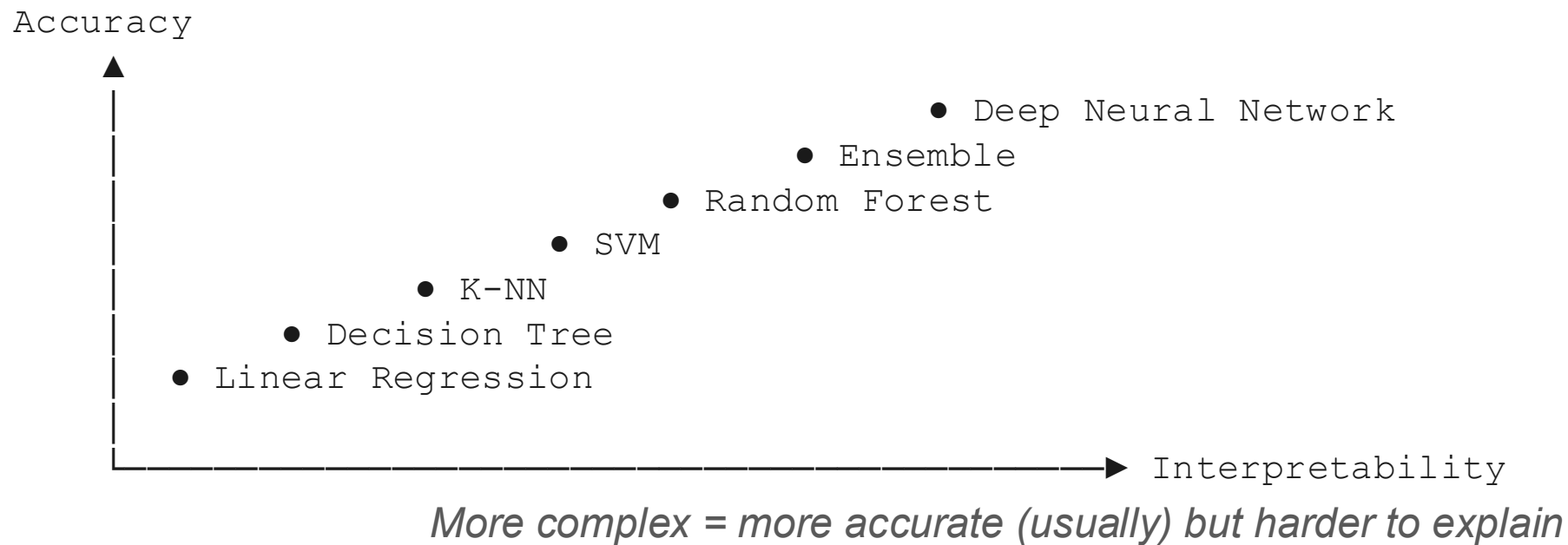
We apply techniques to explain a complex model:

- GradCAM on a CNN
- LIME on a random forest
- SHAP on a neural network

"I need a tool to understand it"

In deep learning, we almost always need explainability (post-hoc) tools.

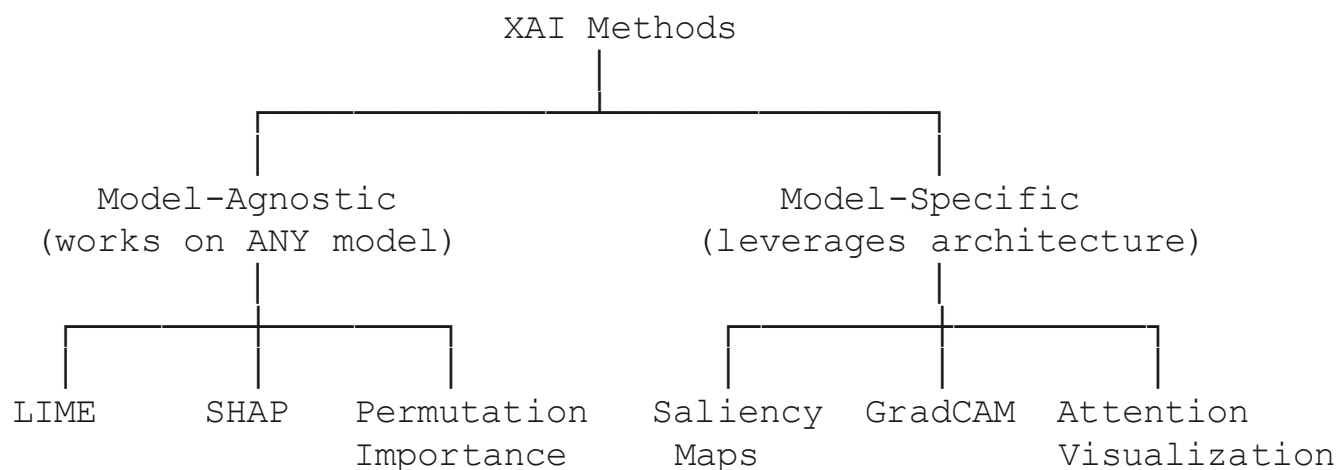
The Interpretability-Accuracy Tradeoff



Key insight:

If a linear model gets 92% and a deep network gets 94%... and the application requires full transparency... the simpler model might be better.

Taxonomy of XAI Methods



Property	Model-Agnostic	Model-Specific
Works on	Any model	Specific architectures
Uses internal structure	No (black-box access)	Yes (gradients, weights)
Speed	Slower (needs perturbation)	Faster (single pass)
Fidelity	Approximate	Can be more faithful
Examples	LIME, SHAP	GradCAM, attention maps

Local vs Global Explanations

LOCAL EXPLANATION

"Why THIS prediction?"



→ "Cat"

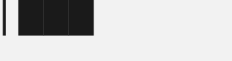
GradCAM highlights
the face, ears, paws

Specific to this image

GLOBAL EXPLANATION

"How does the model work overall?"

Feature Importance:

Ears		0.35
Fur		0.28
Eyes		0.22
Shape		0.15

Aggregated over all images

Today's focus: Mostly local explanations (what does the model see in THIS image?).

3. Model-Agnostic Methods

Explaining Any Model

Permutation Feature Importance

Idea: If a feature is important, shuffling it should hurt performance.

Step 1: Train model, measure baseline acc = 95%

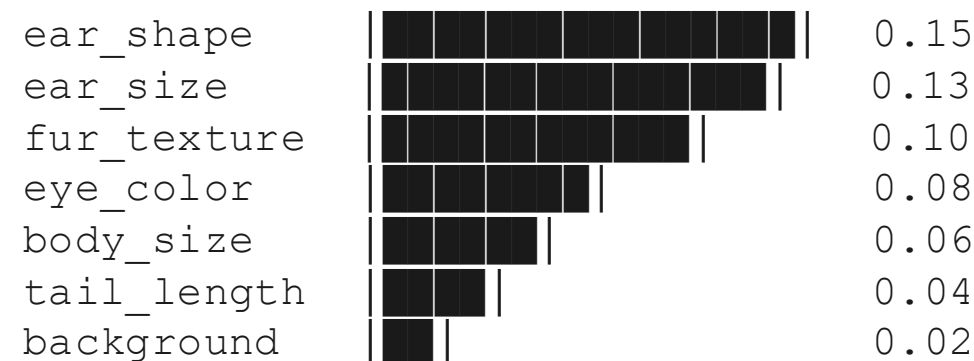
Step 2: Shuffle feature "ear_size" across all test samples (break the relationship between ear_size and label)

Step 3: Re-evaluate → accuracy drops to 82%

Step 4: Importance of ear_size = 95% - 82% = 13%

Step 5: Repeat for all features

Feature Importance (by accuracy drop):



Advantage: Works with any model.

Limitation: Doesn't explain individual predictions; misses feature interactions.

LIME: Local Interpretable Model-agnostic Explanations

Core idea: Approximate the complex model LOCALLY with a simple model.

Step 1: Take the image to explain

Model says: "Cat" (0.94)

Step 2: Generate perturbations

(turn superpixels on/off)

Step 3: Fit a simple (linear) model on perturbations,
weighted by proximity to the original

Step 4: Model coefficients $\rightarrow$ feature importance

Supapixel Approach

Image $\rightarrow$ Supapixel grid (S1..S9)

Generate ~1000 random masks:

Mask 1: [S1, S3, S5] $\rightarrow$ 0.82

Mask 2: [S2, S4, S5] $\rightarrow$ 0.91

Mask 3: [S1, S2, S7] $\rightarrow$ 0.55

Fit: $y \approx w_1 \cdot S1 + w_2 \cdot S2 + \dots + w_9 \cdot S9$

Highest weights $\rightarrow$ most important

Strengths: Intuitive, model-agnostic, human-readable.

Weaknesses: Slow, unstable (varies across runs), superpixel quality matters.

SHAP: SHapley Additive exPlanations

Foundation: Game theory — Shapley values (1953, Nobel Prize).

Analogy: A team of 4 players won a game.
How do you fairly assign credit?

Players: A, B, C, D

Try all coalitions without player A:

$\{\}$ $\rightarrow$ 0	$\{A\}$ $\rightarrow$ 40	A's contribution 40
$\{B\}$ $\rightarrow$ 30	$\{A,B\}$ $\rightarrow$ 50	A's contribution 20
$\{C\}$ $\rightarrow$ 25	$\{A,C\}$ $\rightarrow$ 60	A's contribution 35
$\{B,C\}$ $\rightarrow$ 40	$\{A,B,C\}$ $\rightarrow$ 70	A's contribution 30

Shapley value of A = weighted average
of A's marginal contributions

Applied to ML

"Players" = features (or superpixels)
"Game" = model prediction
"Score" = prediction probability

→ Each feature gets a fair share
of the prediction

SHAP Properties — Why It's Principled

Property	Meaning
Efficiency	Feature contributions sum to the prediction
Symmetry	Equal features get equal attribution
Dummy	Irrelevant features get zero attribution
Additivity	Combined model = combined explanations

Example: Model predicts $P(\text{"Cat"}) = 0.94$

SHAP values:

Base value (average prediction):	0.52
+ Ears contribution:	+0.18
+ Fur texture:	+0.12
+ Eye shape:	+0.08
+ Whiskers:	+0.06
+ Background:	-0.02

= Final prediction:	0.94	✓ Sums up!
---------------------	------	------------

LIME vs SHAP — Quick Comparison

Aspect	LIME	SHAP
Foundation	Local linear approximation	Game theory (Shapley values)
Guarantees	None (heuristic)	Mathematical (4 axioms)
Consistency	Unstable (varies across runs)	Deterministic (given enough samples)
Computation	~1000 perturbations	Exponential (exact) or approximate
Output	Feature weights	Additive feature attributions
Sums to prediction?	No	Yes
Best for	Quick intuition	Rigorous analysis

In practice: SHAP is preferred when rigor matters; LIME when you need quick, visual explanations.

4. Vision-Specific Methods

Leveraging Network Internals

From Model-Agnostic to Model-Specific

Model-agnostic methods treat the network as a black box.

But we built these networks — we know what's inside!

For CNNs, we can exploit:

- |— Gradients (backpropagation signals)
- |— Feature maps (intermediate layer activations)
- |— Weights (learned filters)
- |— Architecture structure (skip connections, attention)

This gives us FASTER, MORE FAITHFUL explanations.

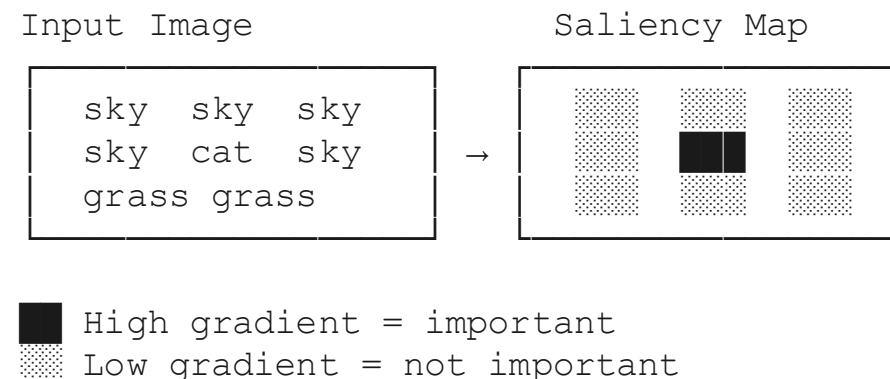
Saliency Maps: The Simplest Gradient Method

Idea: Which input pixels, if changed slightly, would most affect the prediction?

Algorithm:

1. Forward pass: get prediction score for class c
2. Backward pass: compute gradient of score w.r.t. input
3. Take absolute value of gradient
4. Visualize as a heatmap

$$\frac{\partial \text{score}_c}{\partial x_{\text{input}}} \rightarrow \text{saliency map}$$



Strengths: Fast (single backward pass), simple. **Weaknesses:** Noisy, highlights edges rather than semantic regions.

Saliency Map — PyTorch Pseudocode

```
# Vanilla gradient saliency
def compute_saliency(model, image, target_class):
    image.requires_grad_(True)          # Track gradients w.r.t. input

    output = model(image)                # Forward pass
    score = output[0, target_class]      # Score for target class

    score.backward()                    # Backward pass

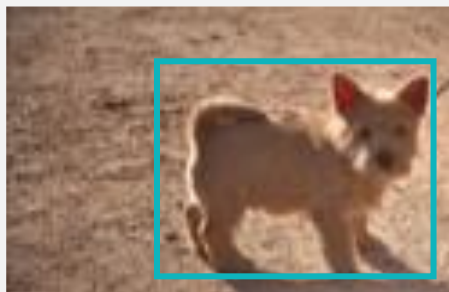
    saliency = image.grad.abs()          # Absolute gradient
    saliency = saliency.max(dim=1)[0]    # Max across RGB channels

    return saliency                     # Shape: (H, W)
```

The Problem with Vanilla Gradients

What we want:

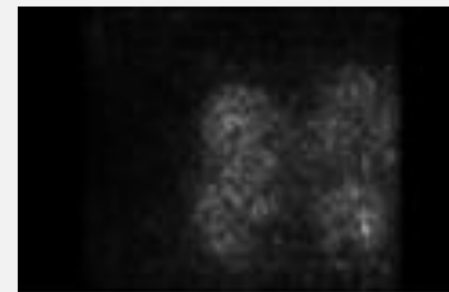
"Which REGION matters?"



Clean, localized

What we often get:

"Which EDGES are there?"



Noisy, scattered

Why? Gradients capture local sensitivity, not semantic importance.

Solutions:

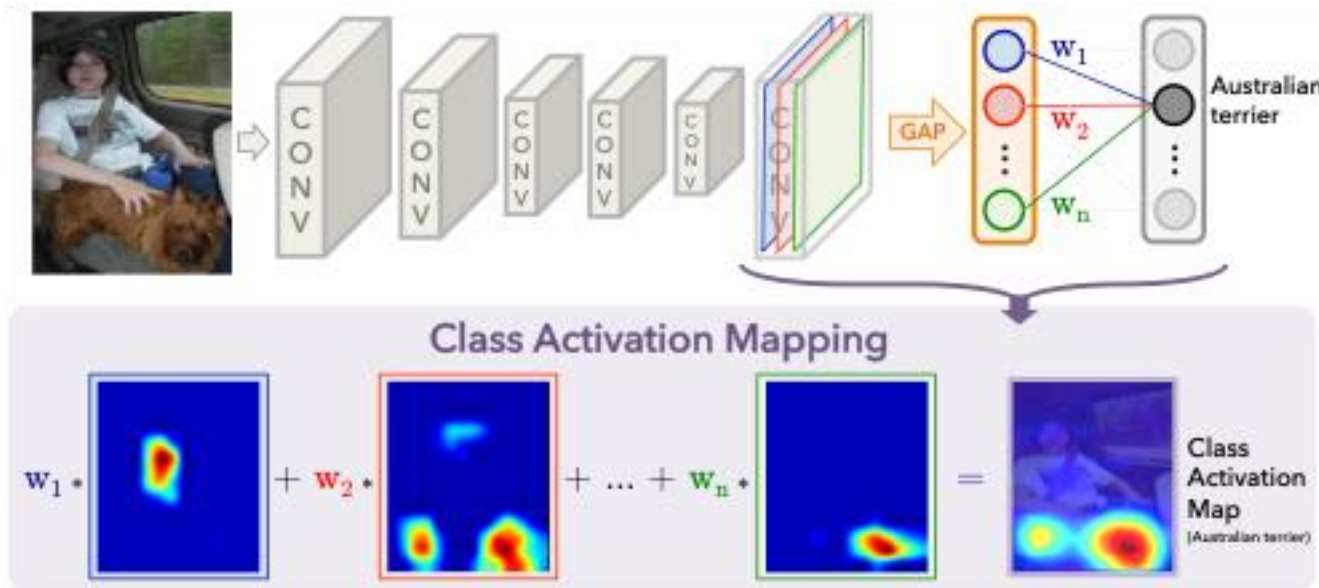
- SmoothGrad: average over noisy copies
- Integrated Gradients: integrate from baseline to input
- GradCAM: use feature maps instead of pixel gradients

Images: arXiv:1312.6034

CAM: Class Activation Mapping

Zhou et al. (2016) — limited to specific architectures.

Requirement: Network must end with: Conv → Global Average Pooling → FC



$$S_c = \sum_k w_k^c \sum_{x,y} f_k(x,y)$$

$$M_c(x,y) = \sum_k w_k^c f_k(x,y)$$

S_c : Score for class c

M_c : Class activation map for class c

$f_k(x,y)$: activation of channel k in the last convolutional layer at (x,y)

w_k^c : classification weight

Limitation: Requires Global Average Pooling → not applicable to all architectures.

GradCAM removes this limitation.

Image: arXiv:1512.04150

GradCAM: Gradient-weighted Class Activation Mapping

Selvaraju et al. (2017) — works on ANY CNN architecture.

The Key Insight

Instead of using FC weights (like CAM), use gradients to compute the importance of each feature map.

$$\alpha_k^c = \frac{1}{Z} \sum_i \sum_j \frac{\partial y^c}{\partial A_{ij}^k}$$

α_k^c = importance weight for feature map k , class c

y^c = score for class c (before softmax)

A^k = activation of feature map k

Z = number of pixels in the feature map ($i \times j$)

GradCAM Algorithm — Step by Step

Step 1: FORWARD PASS

Image $\rightarrow$ CNN $\rightarrow$ class scores
Keep activations A_k from target layer

Step 2: BACKWARD PASS

Compute gradient of class score y^c w.r.t. activations A_k

Step 3: IMPORTANCE WEIGHTS

$\alpha_k^c = \text{GlobalAveragePool}(\partial y^c / \partial A_k)$
(one scalar weight per feature map)

Step 4: WEIGHTED COMBINATION

$L_{\text{GradCAM}} = \text{ReLU}(\sum_k \alpha_k^c \cdot A_k)$

Step 5: UPSAMPLE

Resize to input image dimensions
(bilinear interpolation)

Step 6: OVERLAY

Superimpose heatmap on original image

GradCAM — Visual Walkthrough

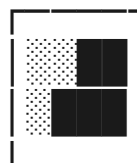
Input Image



→

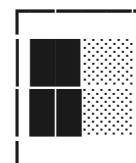
Last Conv Layer (e.g., layer4 in ResNet)

Feature Map 1



$\times \alpha_1$

Feature Map 2



$\times \alpha_2$

... Feature Map 512



$\times \alpha_{512}$

Gradients tell us: $\alpha_1=0.8$, $\alpha_2=0.1$, ... $\alpha_{512}=0.3$

Weighted sum + ReLU → Raw heatmap → Upsample to input size → Overlay on image

Why ReLU in GradCAM?

$$L_{\text{GradCAM}} = \text{ReLU} \left(\sum_k \alpha_k^c \cdot A_k \right)$$

Without ReLU

+++	--	++
--	+++	--
+++	--	++

Includes negative (other classes)

With ReLU

+++	00	++
00	+++	00
+++	00	++

Only POSITIVE contributions

Negative values = pixels that DECREASE the class score (belong to OTHER classes).

ReLU keeps only POSITIVE contributions → "Show me what SUPPORTS the prediction for class c"

GradCAM — PyTorch Implementation

```
class GradCAM:
    def __init__(self, model, target_layer):
        self.model = model
        self.gradients = None
        self.activations = None
        # Register hooks to capture activations and gradients
        target_layer.register_forward_hook(self._save_activation)
        target_layer.register_full_backward_hook(self._save_gradient)

    def generate(self, input_image, target_class):
        output = self.model(input_image) # Step 1: Forward
        self.model.zero_grad()
        output[0, target_class].backward() # Step 2: Backward

        weights = self.gradients.mean(dim=(2,3), keepdim=True) # Step 3: Weights
        cam = (weights * self.activations).sum(dim=1, keepdim=True) # Step 4: Combine

        cam = F.relu(cam) # ReLU!
        cam = F.interpolate(cam, size=input_image.shape[2:], # Step 5: Upsample
                           mode='bilinear', align_corners=False)
        cam = (cam - cam.min()) / (cam.max() + 1e-8) # Normalize
        return cam.squeeze()
```

Using GradCAM — Example

```
import torchvision.models as models

# Load a pretrained ResNet
model = models.resnet50(pretrained=True)
model.eval()

# Target: last convolutional layer
grad_cam = GradCAM(model, model.layer4[-1].conv2)

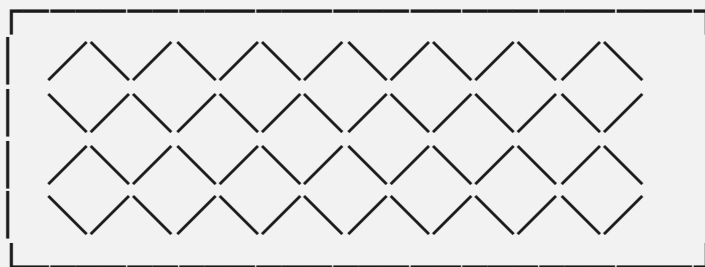
# Generate heatmap for class "tabby cat" (index 281)
heatmap = grad_cam.generate(input_image, target_class=281)

# Overlay on image
plt.imshow(original_image)
plt.imshow(heatmap.numpy(), alpha=0.5, cmap='jet')
plt.title("GradCAM: Where the model sees 'tabby cat'")
plt.show()
```

GradCAM Across Different Layers

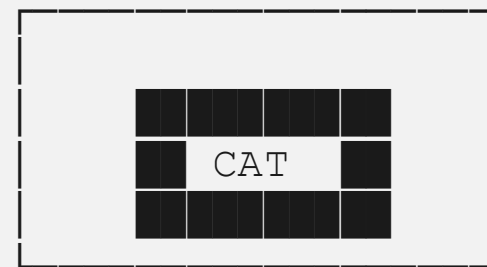
Can we select any layer for explanations?

Early Layers (conv1/layer1)



Fine-grained, edge-level
(hard to interpret)

Late Layers (layer4) ✓



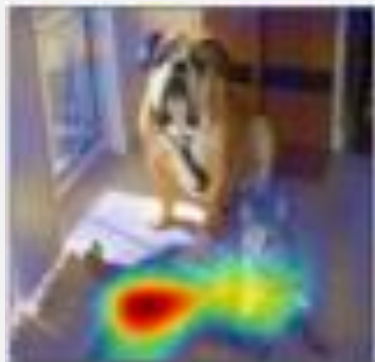
Semantic, object-level
(most useful for XAI)

Best practice: Use the last convolutional layer for the best spatial + semantic balance.

GradCAM for Different Classes

Same image, different target class → different heatmap!

GradCAM for "cat"



GradCAM for "dog"



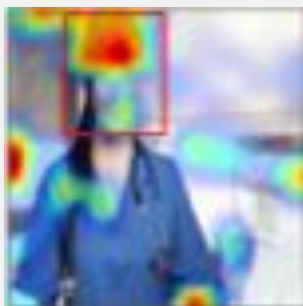
GradCAM is class-discriminative
It explains the specific class, not just "salient regions"

GradCAM for Debugging Models

Case 1: "Right for the wrong reasons"

Prediction: "Nurse" (correct!)

GradCAM reveals:



← Model uses
GENDER,
not uniform!

Bias detected!

Case 2: Correct vs Incorrect predictions

Correct (Cat):

GradCAM → focuses on cat ✓

Incorrect (Cat → Dog):

GradCAM → focuses on background ✗

→ Model is confused by context,
not the actual animal

→ Need more diverse training data

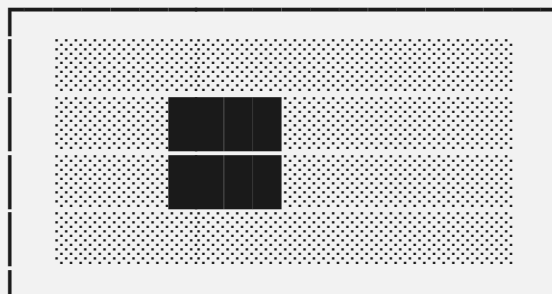
GradCAM Applied to Segmentation

You can apply GradCAM to your U-Net segmentation models!

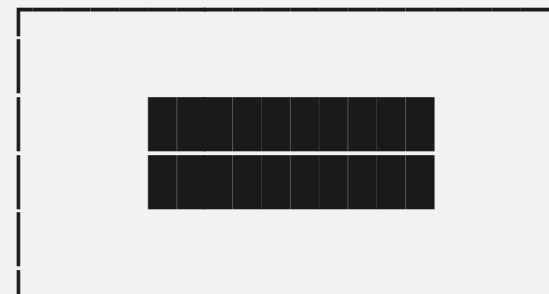
Instead of: "What class is this image?"

Ask: "Why did the model segment this pixel as tumor?"

Segmentation Mask



GradCAM on Decoder



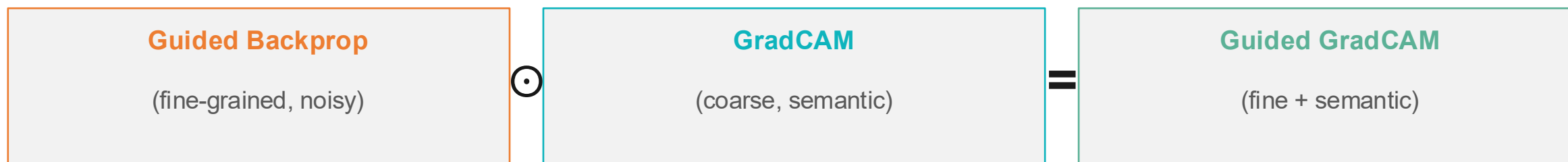
← Encoder features

Useful for: Verifying the model segments based on actual lesion structure, not artifacts.

GradCAM Variants

Method	What It Adds	Trade-off
GradCAM	Gradient-weighted feature maps	Coarse resolution
GradCAM++	Pixel-wise gradient weighting	Better for multiple objects
Guided GradCAM	GradCAM × Guided Backprop	Fine-grained + semantic
Score-CAM	No gradients — uses forward passes	Slower but gradient-free
LayerCAM	Works well on earlier layers too	More faithful at shallow layers

Guided GradCAM: Best of Both Worlds



Attention Visualization

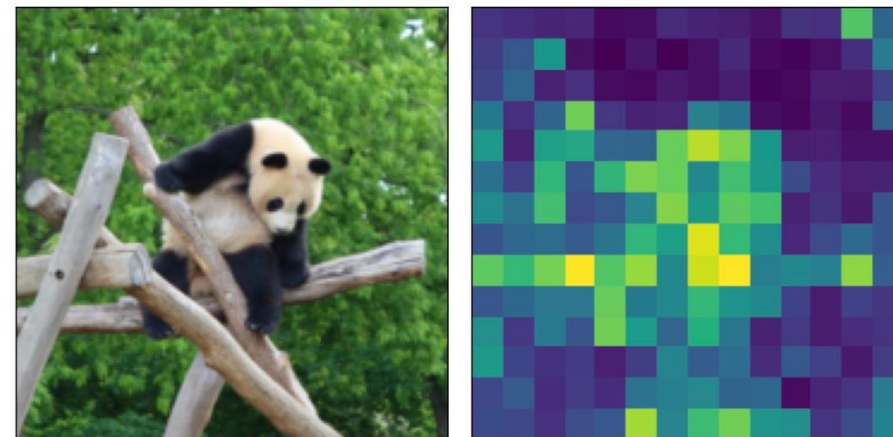
For transformer-based models (ViT, DINO), we can visualize attention maps directly:

Vision Transformer (ViT):

Input Image → Split into patches → Self-attention
across patches

Attention weights tell us:
which patches attend to which?

Patches getting most attention
→ object is there!



<https://alessiodevoto.github.io/vit-attention/>

No gradients needed — attention weights are a natural part of the architecture.

5. Limitations & Critical Thinking

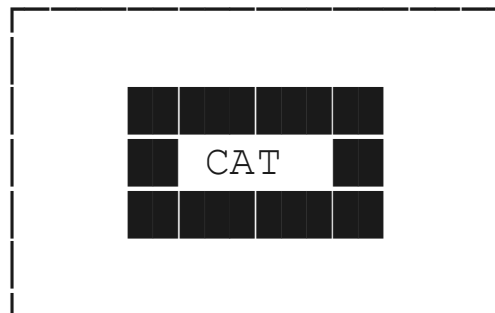
When NOT to Trust Explanations

XAI $\neq$ Trustworthiness

Having an explanation $\neq$ Having a trustworthy model

Model accuracy: 60% (bad model)

GradCAM output:



"See? The model looks at the right region!"

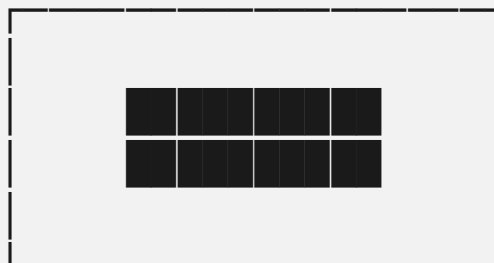
But it still gets 40% of predictions wrong even when looking at the "right" place.

XAI tells you WHERE the model looks, not whether it UNDERSTANDS what it sees.

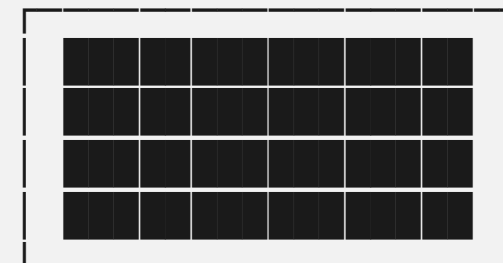
Limitation 1: Explanation Faithfulness

Are explanations truly reflecting the model's decision process?

GradCAM shows



Model might actually use



Texture
patterns
invisible
to heatmap

Post-hoc explanations are APPROXIMATIONS. They highlight correlations, not necessarily causation.

Test: Remove highlighted regions → does the prediction change?
If not, the explanation may not be faithful.

Limitation 2: Human Confirmation Bias

Scenario: Model classifies an X-ray as "Pneumonia". GradCAM highlights the lower right lung.

Doctor A (agrees with diagnosis):

"Yes, that's exactly where I see the opacity. Good model!"

Doctor B (disagrees with diagnosis):

"The model is highlighting normal tissue. Bad explanation."

**SAME explanation, DIFFERENT interpretation.
Humans see what they expect to see.**

The danger:

- Explanations that agree with our intuition feel trustworthy
- Explanations that disagree get dismissed
- Neither response is scientifically rigorous

Mitigation: Use quantitative evaluation, not just visual inspection.

Limitation 3: Adversarial Explanations

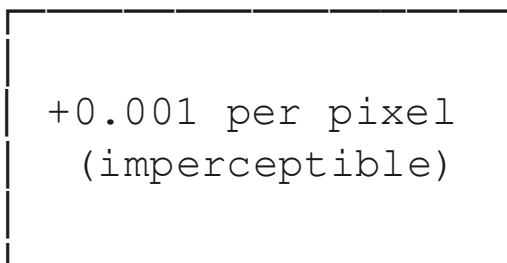
Explanations themselves can be fooled.

Original Image:



+

Adversarial Perturbation:



Result:

Prediction: "Dog" (unchanged)

GradCAM: Completely different heatmap!

The prediction didn't change, but the explanation did.

→ Explanations can be manipulated without affecting predictions

This is a serious concern for high-stakes applications.

Limitation 4: Explanation $\neq$ Causation

GradCAM highlights grass in "cow" predictions. Possible interpretations:

Interpretation A

"Model uses grass to predict cow"
(spurious correlation)

Interpretation B

"Features for cow happen to overlap spatially with grass" (coincidental)

Interpretation C

"Cows are on grass $\rightarrow$ grass is genuinely informative (context)"

Which one is correct? GradCAM alone can't tell you.

XAI reveals correlations in the model's features — determining causation requires controlled experiments (e.g., image editing, counterfactual analysis).

Practical Guidelines: Using XAI Responsibly

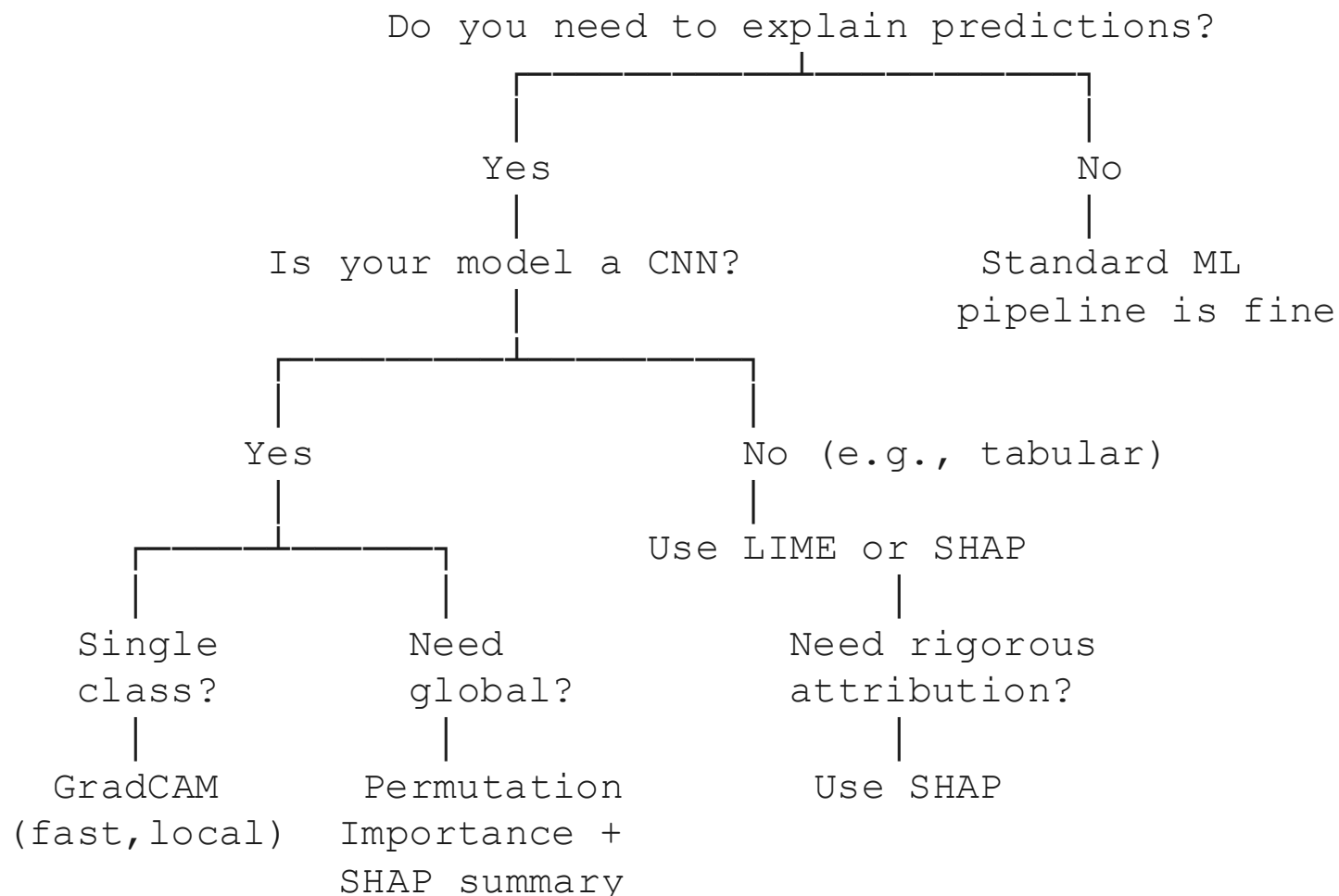
Do

- Use multiple XAI methods and check for consistency
- Compare explanations for correct AND incorrect predictions
- Perform deletion/insertion tests: mask areas, check prediction change
- Document limitations when presenting to stakeholders
- Use XAI for debugging, not just trust

Don't

- Present XAI as proof that a model is correct
- Rely on a single explanation method
- Assume visually pleasing heatmaps mean faithful explanations
- Skip quantitative evaluation of explanation quality
- Use XAI to justify deploying a model that fails standard evaluation

The XAI Decision Framework



Summary

What We Covered Today

What We Covered Today

1. **Why XAI matters** — debugging, trust, compliance, and the Clever Hans effect
2. **The XAI landscape** — model-agnostic (LIME, SHAP) vs model-specific (saliency, GradCAM)
3. **Permutation Importance** — shuffle features, measure accuracy drop
4. **LIME** — local linear approximation via perturbations
5. **SHAP** — game-theoretic, principled attribution
6. **Saliency Maps & GradCAM** — gradient-weighted class-discriminative localization
7. **Limitations** — faithfulness, confirmation bias, adversarial fragility

Key Takeaways

1. XAI is **ESSENTIAL** for responsible AI deployment
2. GradCAM = gradient importance weights $\times$ feature maps
→ fast, class-discriminative, works on any CNN
3. SHAP provides the strongest theoretical guarantees but is expensive for large models
4. No single XAI method is perfect — use multiple methods
5. Explanations can be misleading — always verify quantitatively, not just visually
6. XAI for debugging (development) > XAI for trust (users)

Connecting to Previous Weeks

Model	XAI Application
CNN Classifier (ResNet)	GradCAM: what does the classifier focus on?
Object Detector (YOLO)	GradCAM on backbone: what triggers detections?
U-Net Segmentation	GradCAM on decoder: why this segmentation mask?
All of the above	Systematic analysis with LIME, SHAP, GradCAM

XAI applies to everything you've built so far!

Looking Ahead

Next Week: Attention Mechanisms and Transformers

- From RNNs to attention: solving the sequential bottleneck
- Query, Key, Value: the core attention formulation
- Self-attention and multi-head attention
- Why "Attention Is All You Need" changed everything

Preparation:

- Reading: Vaswani et al. (2017) — "Attention Is All You Need"
- Think about: "How would you handle long-range dependencies in a sequence?"
- Bonus: Watch 3Blue1Brown's attention mechanism video